

高级语言程序设计

实验报告

南开大学 信息安全

姓名：周锐衡

学号：2512504

日期：2026年5月10日

一、作业题目

植物大战僵尸（Plants vs. Zombies）游戏开发

二、开发软件

软件名称	版本	用途
Qt Creator	6.10.2	集成开发环境（IDE）
Qt Framework	6.10.2	图形界面与游戏框架
VS Code	最新版	代码编辑器
GPT-5	2025版	AI辅助开发工具

三、问题描述

1. 项目概述

本项目是一个基于Qt框架开发的《植物大战僵尸》塔防游戏。玩家通过种植各种植物来防御入侵的僵尸，阻止僵尸到达房子。游戏包含完整的植物系统、僵尸系统、卡牌系统、天气系统和游戏平衡机制。

2. 功能需求

植物系统：

植物	功能
向日葵	生产阳光
豌豆射手	发射豌豆攻击僵尸
寒冰射手	发射冰豌豆，减速僵尸
双枪射手	连续发射两颗豌豆
坚果墙	高生命值防御植物

植物	功能
樱桃炸弹	范围爆炸伤害
土豆雷	延时爆炸攻击

僵尸系统：

僵尸	特点
普通僵尸	基础属性
路障僵尸	较高生命值
铁桶僵尸	高生命值
读报僵尸	中等生命值
橄榄球僵尸	高生命值+快速移动

特色功能：

- 天气系统（晴天、雨天、雪天、大风、沙尘暴）
- 命令卡片系统（阳光卡、加速卡）
- 植物升级成长机制
- 动态游戏平衡系统

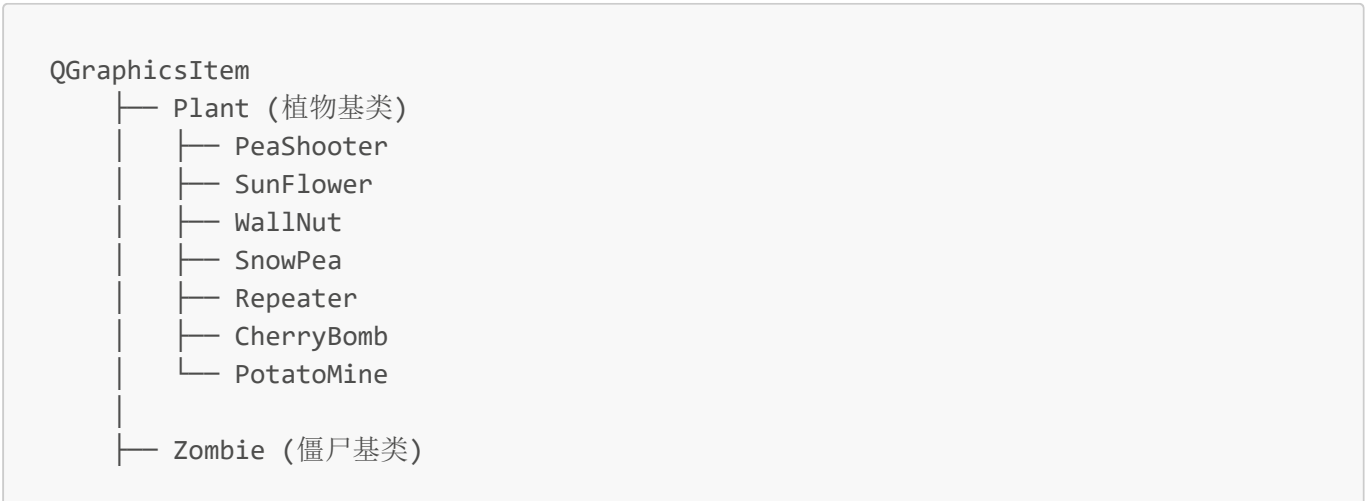
3. 技术难点

- 碰撞检测：**精确判断植物与僵尸、子弹与僵尸的碰撞关系
- 游戏循环：**实现稳定的30FPS游戏主循环
- 动态平衡：**根据玩家等级和倍速实时调整僵尸属性
- 粒子系统：**实现天气粒子的生成与管理
- 状态同步：**确保天气、卡片冷却与游戏倍速同步

四、主要函数

1. 核心架构

类继承关系图：





2. 关键函数实现

2.1 游戏主循环初始化

文件: `mainwindow.cpp`

```
MainWindow::MainWindow(QWidget *parent) : QMainWindow(parent)
{
    timer = new QTimer;
    scene = new QGraphicsScene(this);
    scene->setSceneRect(150, 0, 900, 600);

    connect(timer, &QTimer::timeout, scene, &GGraphicsScene::advance);
    connect(timer, &QTimer::timeout, this, &MainWindow::addZombie);
    connect(timer, &QTimer::timeout, this, &MainWindow::check);

    timer->start(33); // 约30FPS
}
```

功能说明:

- 创建33ms定时器，实现约30FPS的游戏帧率
- 连接定时器信号到场景更新、僵尸生成和胜利检测函数

2.2 碰撞检测实现

文件: `plant.cpp`

```
bool Plant::collidesWithItem(const QGraphicsItem *other, Qt::ItemSelectionMode mode) const
{
    Q_UNUSED(mode)
    return other->type() == Zombie::Type &&
        qFuzzyCompare(other->y(), y()) &&
        qAbs(other->x() - x()) < 30;
}
```

功能说明:

- 检测植物是否与僵尸发生碰撞
- 使用qFuzzyCompare处理浮点数精度问题

2.3 僵尸生成逻辑

文件：mainwindow.cpp

```
void MainWindow::addZombie()
{
    static int gameFrame = 0;
    gameFrame++;

    int bucketFrame = 60 * 1000 / 33;    // 1分钟
    int screenFrame = 2 * 60 * 1000 / 33; // 2分钟
    int footballFrame = 3 * 60 * 1000 / 33; // 3分钟

    if (gameFrame < bucketFrame) {
        zombie = (rand < 60) ? new ConeZombie : new BasicZombie;
    } else if (gameFrame < screenFrame) {
        if (rand < 10)      zombie = new BucketZombie;
        else if (rand < 60) zombie = new ConeZombie;
        else                zombie = new BasicZombie;
    }
}
```

僵尸刷新规则：

时间阶段	出现僵尸类型
0-1分钟	普通僵尸、路障僵尸
1-2分钟	增加铁桶僵尸
2-3分钟	增加读报僵尸
3分钟后	增加橄榄球僵尸

2.4 植物升级系统

文件：wallnut.cpp

```
void WallNut::advance(int phase)
{
    if (!phase) return;

    if (hp < lastHp) {
        int damageTaken = lastHp - hp;
        if (damageTaken > 0) {
            gainExperience(damageTaken / 50);
        }
    }
}
```

```
        lastHp = hp;
    }

    if (++counter >= CommandManager::instance()->effectiveTicksFor(time)) {
        counter = 0;
        gainExperience(4); // 被动经验获取
    }
}
```

坚果墙升级机制：

项目	数值
被动经验	每60帧获得4点经验
受伤经验	damageTaken / 50（取整）
1→2级所需经验	500点
2→3级所需经验	1500点
升级效果	生命值×1.5，回复满血

2.5 子弹天气影响

文件：pea.cpp

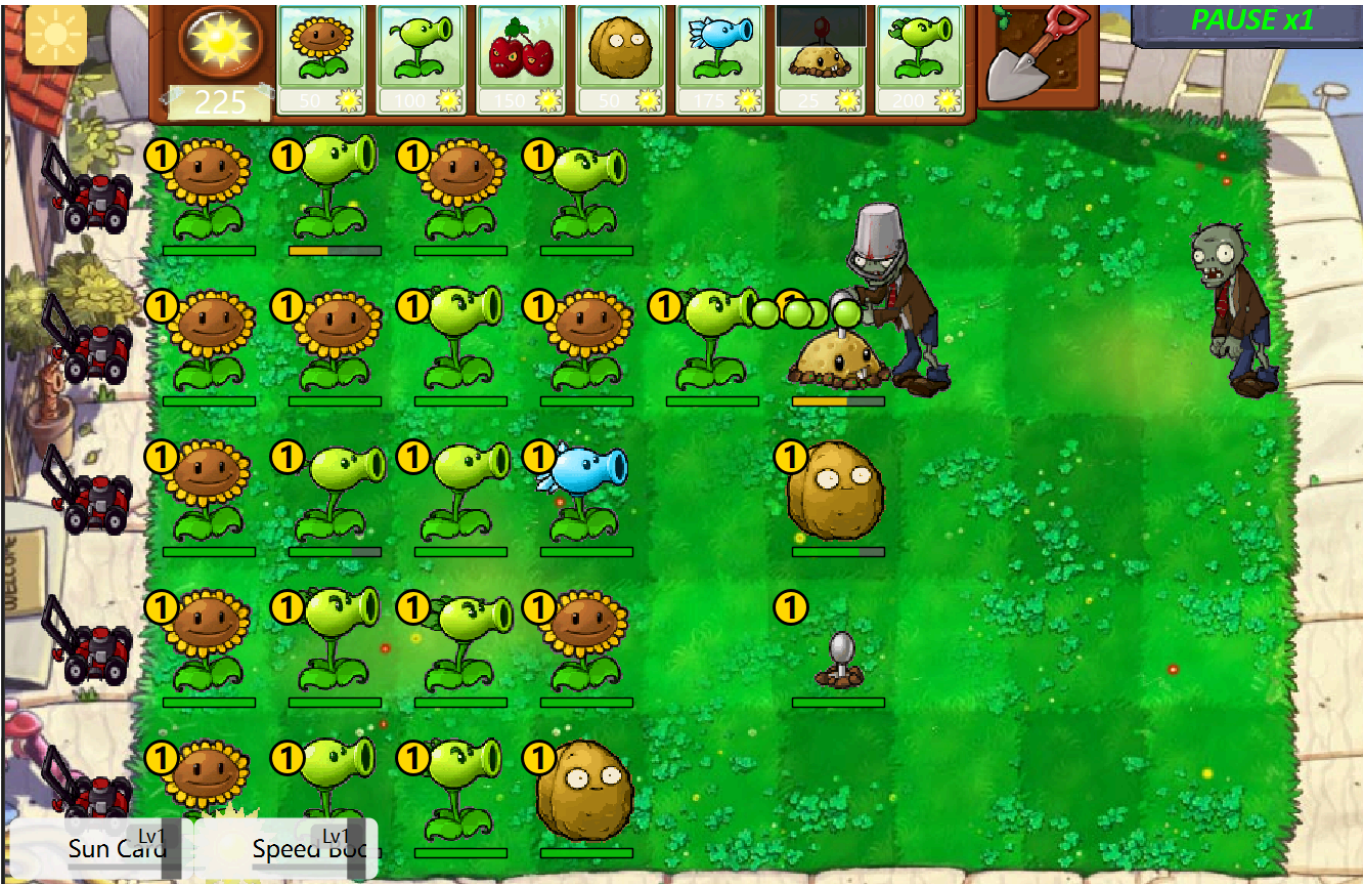
```
void Pea::advance(int phase)
{
    qreal factor = 1.0;
    switch (WeatherManager::instance()->current())
    {
        case Weather::Wind:    factor = 1.3; break;
        case Weather::Snow:    factor = 0.75; break;
        case Weather::Sandstorm: factor = 0.65; break;
    }

    setX(x() + speed * factor * CommandManager::instance()->speedMultiplier());
}
```

3. 设计模式应用

设计模式	应用场景	具体实现
单例模式	WeatherManager、CommandManager	全局唯一实例管理
模板方法	Plant::advance()	子类重写特定行为
工厂模式	Zombie生成	根据类型创建不同僵尸
观察者模式	天气变化通知	信号槽机制实现

五、游戏截图



界面说明：

- 左上角：阳光计数器（显示当前阳光数量）
- 顶部：植物卡牌栏（向日葵、豌豆射手、樱桃炸弹等）
- 中间：5×9的草坪格子区域
- 底部：命令卡片（阳光卡、加速卡）
- 右上角：暂停按钮和倍速显示

六、单元测试

1. 测试用例表

测试编号	测试功能	输入条件	预期结果	测试结果
TC001	豌豆射手攻击	放置豌豆射手，生成僵尸	僵尸血量减少，豌豆消失	☑ 通过
TC002	向日葵产阳光	放置向日葵，等待240帧	生成阳光道具	☑ 通过
TC003	坚果墙升级	攻击坚果墙至500经验	坚果墙升级至2级，HP恢复	☑ 通过
TC004	僵尸生成时间	游戏运行1800帧（约1分钟）	生成铁桶僵尸	☑ 通过
TC005	天气效果	切换到雨雪天气	背景变色，粒子效果生成	☑ 通过
TC006	倍速同步	使用2倍速	僵尸移动、卡片冷却同步加快	☑ 通过
TC007	碰撞检测	僵尸接近植物	触发攻击/啃食动画	☑ 通过

测试编号	测试功能	输入条件	预期结果	测试结果
TC008	游戏失败检测	僵尸到达左侧	显示失败画面，游戏停止	✅ 通过

2. 测试环境

- CPU: Intel Core i7-10700K
- 内存: 16GB
- 操作系统: Windows 11
- Qt版本: 6.10.2

七、调试过程

1. 问题记录

问题1：僵尸生成时间不准确

现象：游戏运行数分钟后仍未出现高级僵尸

原因：gameFrame变量在addZombie()内部定义，仅在生成时递增

解决方案：将gameFrame改为静态变量，每次函数调用都递增

```
static int gameFrame = 0;
gameFrame++; // 每次调用都递增
```

问题2：坚果墙升级过快

现象：坚果墙被攻击后瞬间从1级升至3级

原因：checkLevelUp()使用while循环

解决方案：将while改为if，每次只升级一级

```
if (level < 3 && experience >= expRequired[level]) {
    ++level;
    hp = getMaxHp();
}
```

问题3：经验计算错误

现象：坚果墙每秒获得大量经验

原因：lastHp = maxHp在每帧执行

解决方案：仅在受伤后更新lastHp

问题4：语法错误

现象： QFont类型没有重载operator->

原因： font是对象而非指针

解决方案：使用 . 操作符替代->

```
font.setBold(true); // 而非 font->setBold(true)
```

八、游戏平衡参数

初始设置

参数	数值
初始阳光	150
阳光卡效果	200阳光
天空掉落阳光	25/个
卡片升级费用	1000阳光

植物属性

植物	生命值	攻击力/冷却
向日葵	300	10秒/次
豌豆射手	300	25伤害/1.4秒
坚果墙	4000/6000/8000 (Lv1/Lv2/Lv3)	-

僵尸属性

僵尸	生命值	攻击力
普通	200	1
路障	480	1
铁桶	1030	1
读报	1030	1
橄榄球	1250	1

九、收获

1. 技术收获

- 1. **Qt框架掌握：** 深入理解Qt Graphics View框架的使用
- 2. **面向对象设计：** 学会使用继承、多态等OOP特性

3. 设计模式应用：掌握单例模式、观察者模式等常用设计模式
4. 游戏开发基础：理解游戏主循环、帧率控制、状态同步等核心概念

2. AI工具使用披露

AI工具名称：GPT-5（2025版）

使用用途：

- 代码生成：辅助编写植物、僵尸等类的实现代码
- 问题诊断：分析程序运行时错误原因
- 设计建议：提供架构设计和优化建议
- 文档编写：辅助撰写项目文档和注释

使用位置：

- plant.cpp：植物基类实现
- zombie.cpp：僵尸基类实现
- wallnut.cpp：坚果墙升级系统
- pea.cpp：子弹飞行逻辑
- weather.cpp：天气系统实现

十、附录

项目文件结构

```
pvz/
├── main.cpp           # 程序入口
├── mainwindow.cpp    # 主窗口和游戏主循环
├── plant.h/cpp       # 植物基类
├── zombie.h/cpp      # 僵尸基类
├── peashooter.h/cpp  # 豌豆射手
├── sunflower.h/cpp   # 向日葵
├── wallnut.h/cpp     # 坚果墙
├── cherrybomb.h/cpp  # 樱桃炸弹
├── potatomine.h/cpp  # 土豆雷
├── snowpea.h/cpp     # 寒冰射手
├── repeater.h/cpp    # 双枪射手
├── basiczombie.cpp   # 普通僵尸
├── conezombie.cpp    # 路障僵尸
├── bucketzombie.cpp  # 铁桶僵尸
├── screenzombie.cpp  # 读报僵尸
├── footballzombie.cpp # 橄榄球僵尸
├── shop.cpp          # 商店系统
├── card.cpp          # 卡牌系统
├── weather.cpp       # 天气系统
├── commandcard.cpp   # 命令卡系统
├── commandmanager.cpp # 命令管理器
├── pea.h/cpp         # 豌豆子弹
└── sun.h/cpp         # 阳光
```

└─ images/	# 图片资源
└─ PVZ.pro	# Qt项目文件

运行方式

1. 使用Qt Creator打开PVZ.pro
2. 配置编译环境（Qt 6.10.2 + MSVC 2022）
3. 编译运行项目
4. 点击开始按钮启动游戏